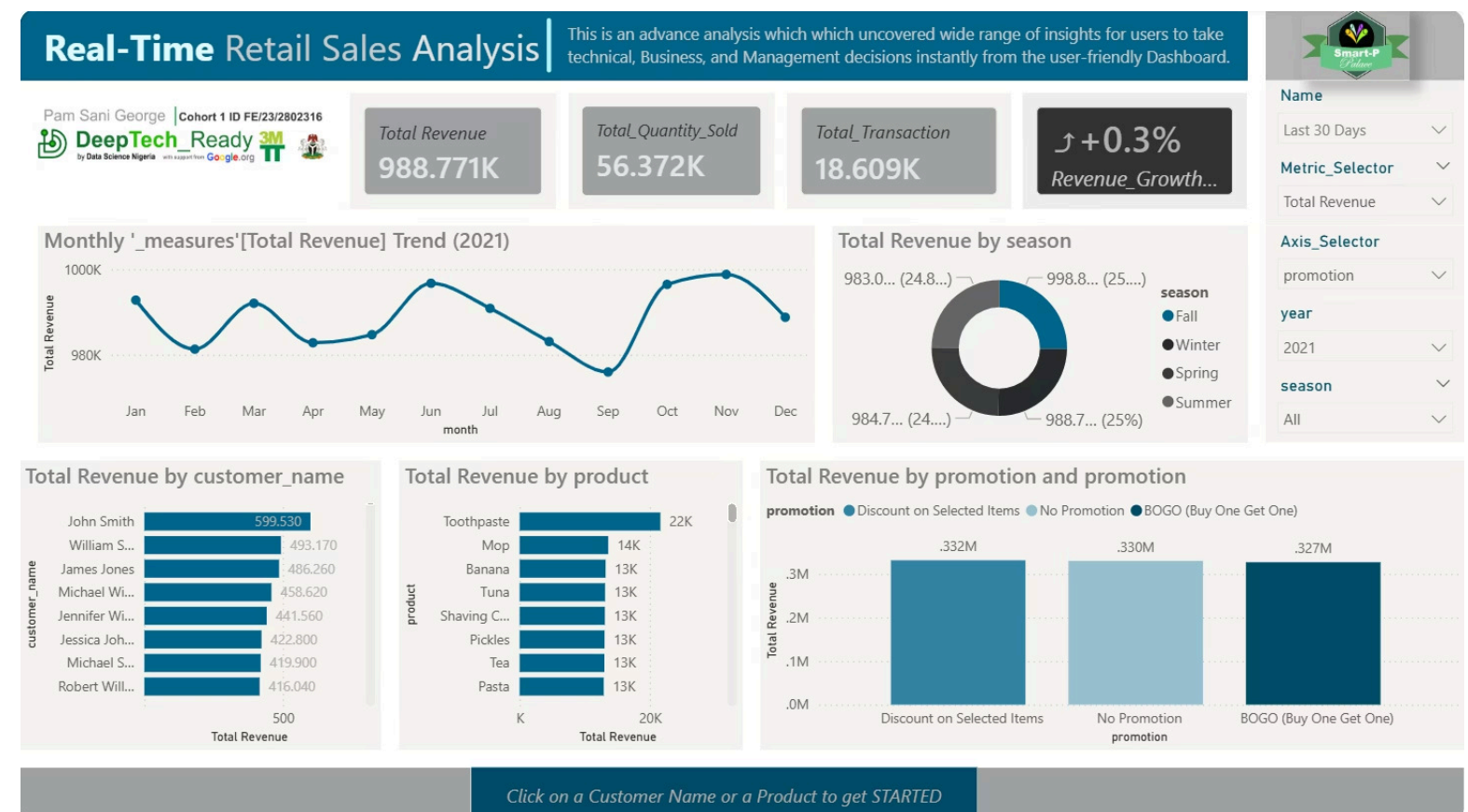
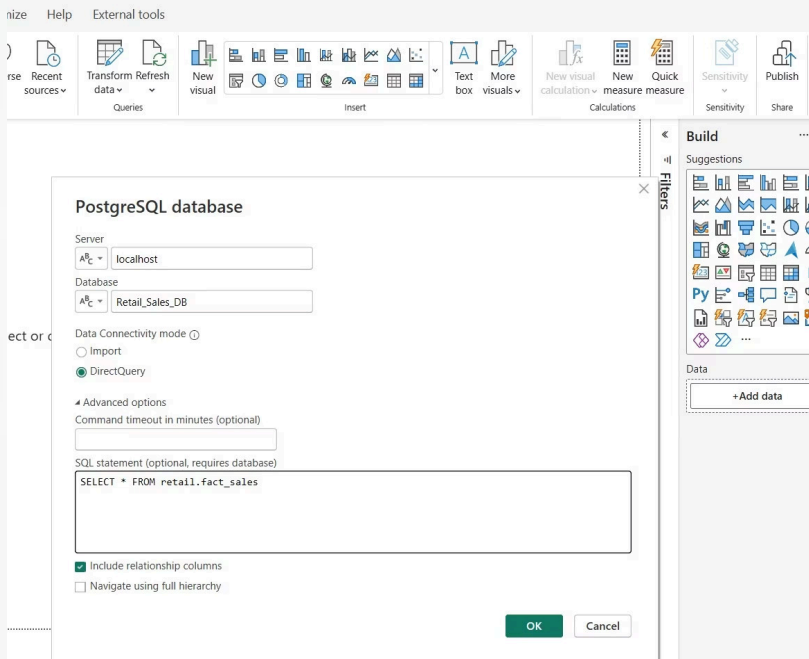


Real-Time Retail Sales Analysis — Capstone Project



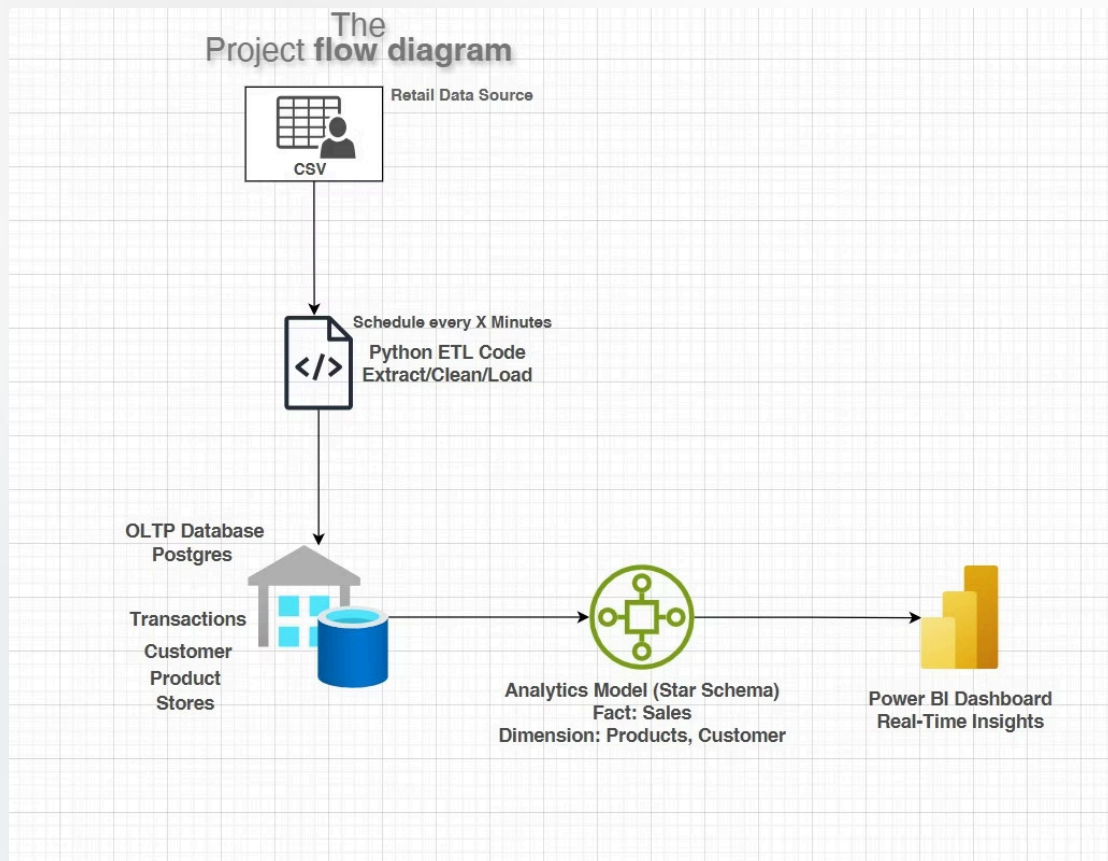
End-to-end build of a real-time retail analytics pipeline: from raw CSVs to a production-ready Power BI dashboard. Focus: engineering, data integrity, and actionable KPIs for stakeholders and analytics hiring managers.



Project Objective

Turn raw CSV data into operational insights

Transform diverse retail CSV datasets into a validated relational schema, load into PostgreSQL, enable DirectQuery and model refinement in Power BI, and deliver a dashboard that supports real-time decision-making across marketing, inventory and store operations.



Scope & Success Criteria

Data Quality

Complete cleaning, deduplication, type enforcement and referential integrity checks.

Performance

Optimised PostgreSQL schema and DirectQuery patterns for sub-second KPI refresh where possible.

Usability

Intuitive dashboard with drillthroughs, filters and explanation of assumptions.

Business Value

Deliver insights that drive promotion optimisation, inventory decisions and targeted customer campaigns.

Tools & Technologies



Python

Data extraction, cleaning, transformation (pandas), and CSV processing pipelines.



PostgreSQL

Relational storage, indexing, constraints, and integrity checks for production datasets.



SQL

Data validation, complex joins, window functions for cohort and trend calculations.



Power BI

Data modelling (calculation groups), DirectQuery, visualisation and drillthrough UX.



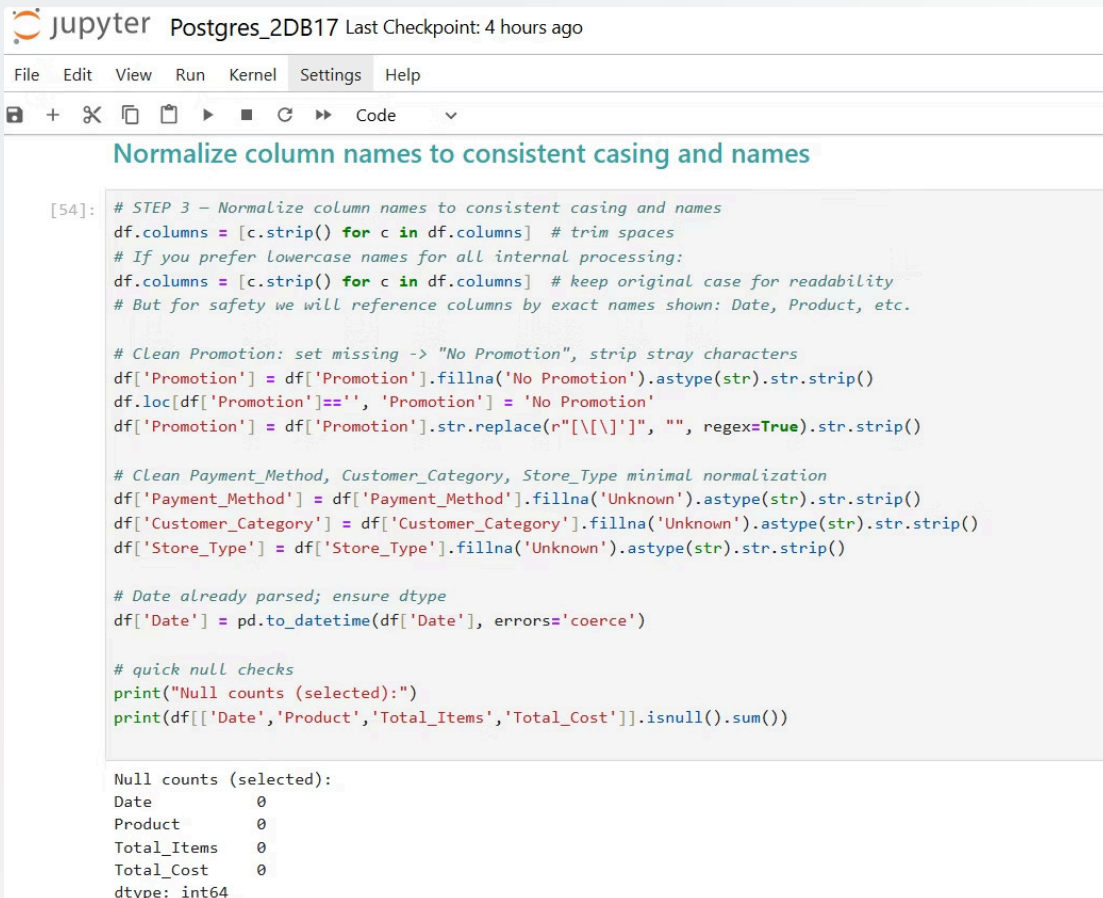
Tabular Editor 2

Advanced model management: calculation groups, measure templates and deployment automation.



CSV Datasets

Sales, transactions, promotions, customers and product reference files — source of truth for ETL.



The screenshot shows a Jupyter Notebook titled "Postgres_2DB17" with a last checkpoint of 4 hours ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The active cell is a code cell with the following Python code:

```
[54]: # STEP 3 - Normalize column names to consistent casing and names
df.columns = [c.strip() for c in df.columns] # trim spaces
# If you prefer lowercase names for all internal processing:
df.columns = [c.strip() for c in df.columns] # keep original case for readability
# But for safety we will reference columns by exact names shown: Date, Product, etc.

# Clean Promotion: set missing -> "No Promotion", strip stray characters
df['Promotion'] = df['Promotion'].fillna('No Promotion').astype(str).strip()
df.loc[df['Promotion']=='', 'Promotion'] = 'No Promotion'
df['Promotion'] = df['Promotion'].str.replace(r"[\[\]]", "", regex=True).str.strip()

# Clean Payment_Method, Customer_Category, Store_Type minimal normalization
df['Payment_Method'] = df['Payment_Method'].fillna('Unknown').astype(str).strip()
df['Customer_Category'] = df['Customer_Category'].fillna('Unknown').astype(str).strip()
df['Store_Type'] = df['Store_Type'].fillna('Unknown').astype(str).strip()

# Date already parsed; ensure dtype
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')

# quick null checks
print("Null counts (selected):")
print(df[['Date', 'Product', 'Total_Items', 'Total_Cost']].isnull().sum())
```

The output of the code cell is a table showing null counts for selected columns:

Null counts (selected):	
Date	0
Product	0
Total_Items	0
Total_Cost	0
dtype: int64	

Data Preparation & Upload (Python)

Workflow: import CSVs → standardise column types → normalise dates/currencies → explode nested fields (JSON in columns) → deduplicate and enrich (geography, store type) → create normalized relational tables (customers, products, transactions, promotions) → bulk upsert to PostgreSQL using COPY and parameterised transactions.

Key checks

- Null-rate thresholds
- Type sanity (dates, numeric ranges)
- Foreign key existence

Optimisations

- Chunked processing for large CSVs
- Vectorised transforms (pandas / pyarrow)
- Parallel uploads with COPY

Database Setup (PostgreSQL)

The screenshot displays a PostgreSQL database management interface. On the left, a tree view shows the database structure for 'Retail_Sales_DB', including Schemas (public, retail), Tables (7), and various database objects like Aggregates, Collations, Domains, FTS Configurations, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, and Tables. The 'Tables' section is expanded, showing a list of dimension tables: dim_customer, dim_date, dim_payment_method, dim_product, dim_promotion, and dim_store.

The main window shows a query editor with the following SQL query:

```
---High-Value Customers (Spending ≥ 10,000)---
SELECT
  c.customer_name,
  SUM(f.revenue) AS total_spent
FROM retail.fact_sales f
JOIN retail.dim_customer c
  ON f.customer_id = c.customer_id
GROUP BY c.customer_name
HAVING SUM(f.revenue) >= 10000
ORDER BY total_spent DESC
LIMIT 10;
```

The query results are displayed in a table with 10 rows, showing the top high-value customers. The columns are 'customer_name' (text) and 'total_spent' (numeric).

	customer_name	total_spent
1	Michael Smith	23971.39
2	James Smith	18505.92
3	Michael Johnson	18128.08
4	Christopher Smith	17069.07
5	David Smith	16686.67
6	Michael Williams	15434.11
7	Michael Jones	15253.65
8	Jennifer Smith	14518.32
9	John Smith	14349.69
10	Michael Brown	14231.64

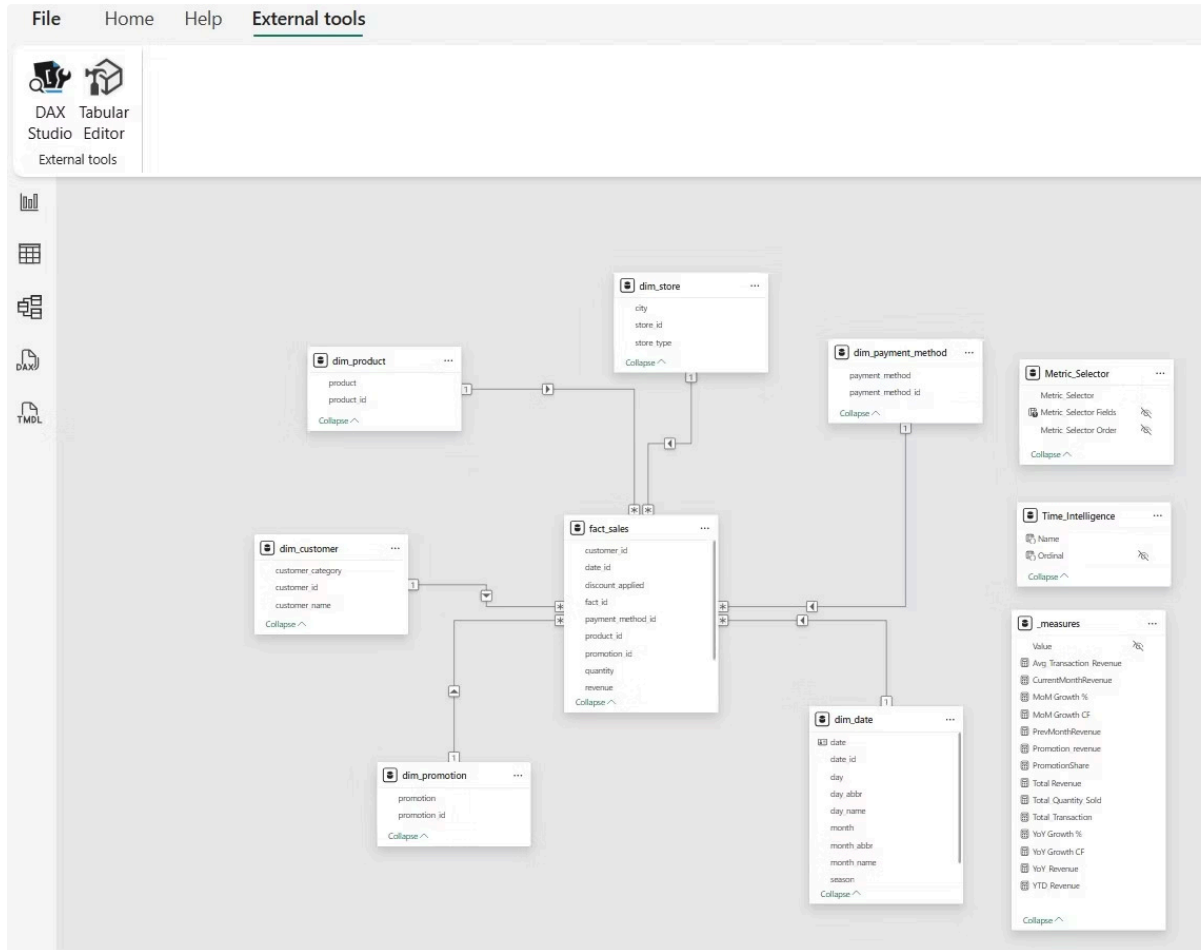
The bottom status bar indicates 'Total rows: 10' and 'Query complete 00:00:57.898'.

Schema design: star-like analytical model with fact_sales and dimension tables (dim_customer, dim_product, dim_store, dim_promotion). Implemented:

- Primary keys, composite keys for transactional uniqueness
- Foreign-key constraints and deferred checks for bulk loads
- Indexes for high-cardinality columns (customer_id, product_id, transaction_date)

Integrity queries used for validation: referential integrity, row counts per source, and checksum comparisons.

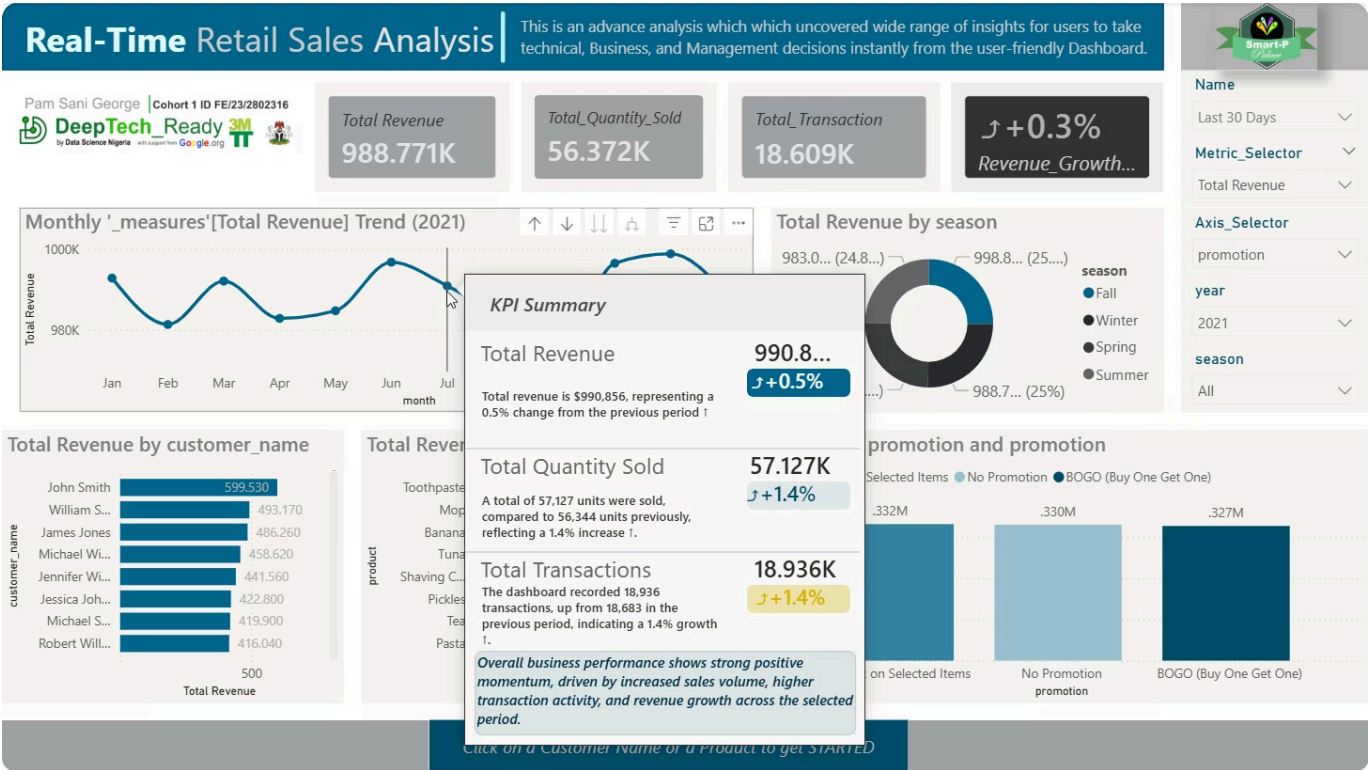
Power BI — Data Integration & Modelling



Integration approach:

- DirectQuery for near-real-time metrics; import mode for large static reference tables.
- Define relationships and cardinality; enforce single-direction filter flow where appropriate.
- Use Tabular Editor to create calculation groups (Time intelligence & Promo adjustments) and standardised measure templates.
- Parameterise connections for environment promotion (dev → staging → prod).

Dashboard Design

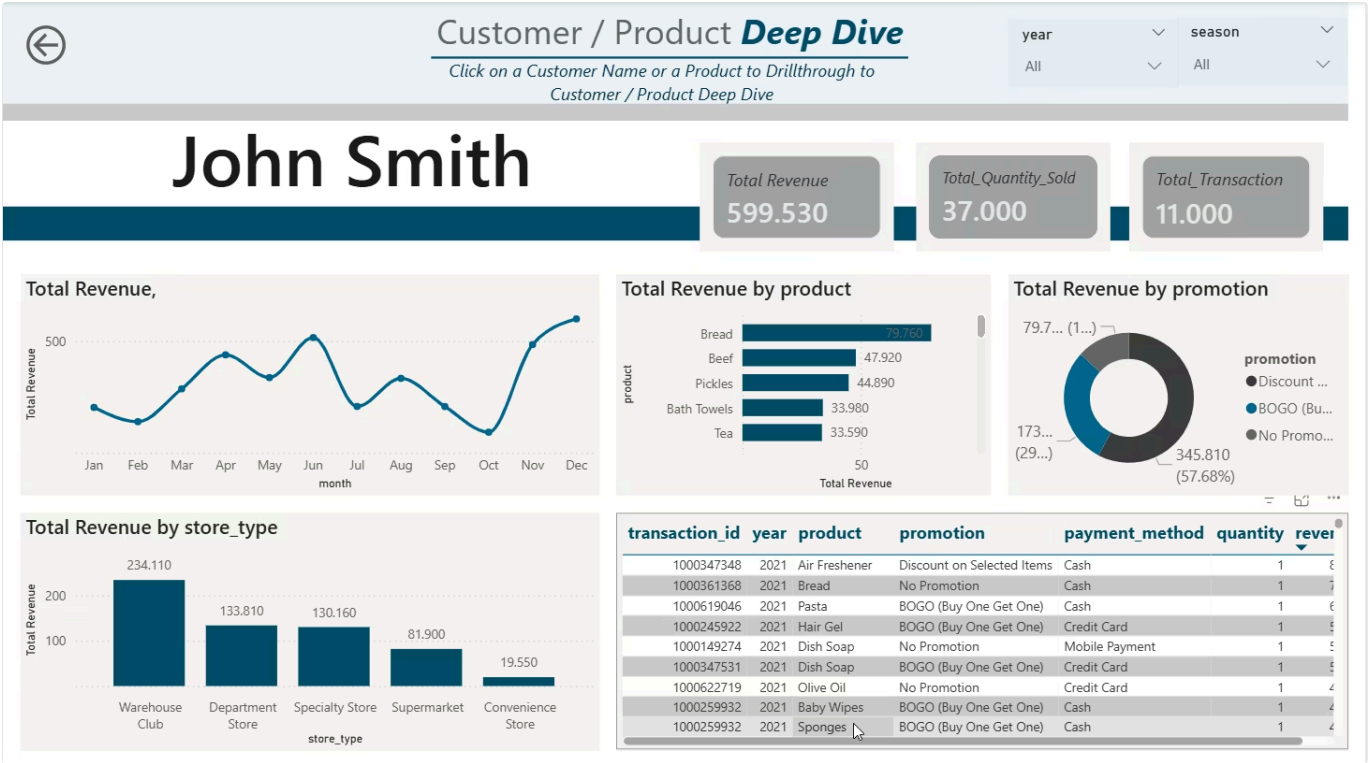


Main page components:

- KPI banner: revenue, daily active customers, average basket value (ABV), margin %.
- Seasonality panel: weekly & monthly distribution heatmaps.
- Promotion effectiveness: BOGO vs discount lift visualisation with incremental revenue metrics.
- Store-type comparison: performance by store-format and region.
- Top products & customers grid with concentration metrics (Pareto analysis).

Drillthrough page: customer lifecycle view, product SKU timelines, and promotion-level attribution filters for diagnostic analysis.

Key Insights & Visualisations



Business Impact & Next Steps



Appendix includes Python scripts, PostgreSQL DDL, and Power BI model notes – placeholders prepared for final screenshots and code snippets.

Strategic Planning

Real-time KPIs enable data-driven promo calendars and dynamic pricing tests.

Customer Segmentation

Use revenue concentration and behaviour cohorts to personalise offers and reduce churn.

Inventory Optimisation

Seasonal signals inform replenishment cadence and safety stock adjustments.

Operationalise

Automate nightly/near-real-time ETL, add alerting for anomalies, and promote model to production with documented runbooks.